



Resumen de avance

Desde la propuesta visual con Claude hasta la implementacion de login, roles, CRUDs y cedulas arbitrales.

Estado general
El proyecto avanza de una prueba visual/tabla inicial a una base MVC funcional con conexión SQL Server, design system, CRUDs principales, módulo de partidos/cedulas y autenticación básica por roles.

Dato	Descripcion
Proyecto	SIGELFA - Sistema Gestor de Liga de Futbol Amateur
Fecha del resumen	26 de mayo de 2026
Rama / flujo sugerido	Trabajar por feature branches y PR hacia main
Alcance del PDF	Resumen desde el design system hasta login/roles

1. Resumen ejecutivo

Desde que se pidio a Claude una propuesta visual, el proyecto SIGELFA se movio de una interfaz inicial con tabla de equipos a una aplicacion PHP + SQL Server con arquitectura MVC sencilla, componentes visuales premium y modulos administrativos funcionales. El color principal definido fue #09a400, con base clara #F8F9FA y un sidebar oscuro #141619.

La direccion visual se implemento bajo el concepto **Administrative Dark-Trim**: una interfaz clara con sidebar oscuro, cards, tablas, botones compactos, badges, sombras suaves y animaciones discretas. Despues se construyeron CRUDs MVC para Equipos, Jugadores y Arbitros, se integro el modulo inicial de Partidos/Cedulas Arbitrales y se agrego autenticacion basica con roles de aplicacion.

Punto clave

El avance ya conecta directamente con la practica de cedulas arbitrales: existe login base, catalogo de arbitros, listado de partidos, captura de cedulas y uso de stored procedures con transaccion.

2. Linea de tiempo del avance

Paso	Bloque	Resultado
1	Design system con Claude	Se definio concepto visual, paleta, tokens, tipografia, sidebar, tablas, forms, cards, badges y animaciones.
2	Base visual con Codex	Se implemento el estilo premium en CSS, layouts, topbar/sidebar, tabla de equipos y JS de UI.
3	Ajustes de UX	Se quito el badge "Local", se corrigio overflow global, se compacto la tabla y se elimino Desactivar del listado.
4	CRUD Equipos	Se creo MVC para listar, crear, ver, editar y manejar estado activo/inactivo.
5	CRUD Jugadores	Se agrego MVC con generacion automatica de numJug, relacion con Equipo y Posicion.
6	CRUD Arbitros	Se agrego MVC con generacion automatica de numArb y catalogo base para cedulas.
7	Partidos y Cedulas	Se agrego modulo MVC para listar partidos, capturar cedulas y agregar eventos con stored procedures.
8	Login y roles	Se agregaron tablas Usuario/UsuarioArbitro, AuthController, helper auth, login y proteccion de rutas.

3. Diseno visual aprobado

La propuesta visual recibida se baso en un estilo minimalista y premium. El verde #09a400 se usa como acento de identidad, no como fondo general. El sistema evita verse generico o escolar, y busca una experiencia cercana a dashboards administrativos modernos.

Elemento	Decision
Concepto	Administrative Dark-Trim: interfaz clara con sidebar oscuro como ancla visual.
Color principal	#09a400 para CTA, estado activo, focus ring, item activo y acentos.
Base clara	#F8F9FA y #f0f2f5 para fondo de app y superficies.
Sidebar	#141619, fijo/estable como elemento de navegacion principal.

Elemento	Decision
Tipografia	DM Sans para interfaz y DM Mono para IDs/codigos.
Componentes	Botones compactos, cards con sombra, tablas premium, badges y formularios.
Animaciones	fadeUp, shimmer y transiciones de 150ms con cubic-bezier(.4,0,.2,1).

4. Implementacion de la base visual

Codex implemento la base visual dentro del proyecto, respetando la ejecucion oficial con `php -S localhost:8000 -t public`, sin depender de Laragon y sin tocar la conexion ni archivos sensibles.

- Se actualizo **public/assets/css/style.css** con tokens, layout, sidebar, topbar, botones, cards, tablas, badges, forms, alertas, empty states y responsive.
- Se creo **public/assets/js/ui.js** para interacciones ligeras como el sidebar movil.
- Se actualizaron layouts: **header.php**, **sidebar.php** y **footer.php**.
- Se mantuvo la salida dinamica escapada con **h()** o **htmlspecialchars**.
- Se corrigieron problemas de overflow: app-shell flex, sidebar con ancho fijo, main con min-width: 0, tablas compactas y sin scroll global.

5. CRUD de Equipos

El primer CRUD real fue Equipos, porque era el punto de entrada natural para validar el patron MVC, la conexion a SQL Server y la base visual.

Archivo / area	Cambio principal
public/index.php	Paso a ser front controller simple para enrutar controller/action.
EquipoController.php	Acciones index, create, store, show, edit, update y manejo de flash/validaciones.
Equipo.php	Metodos obtenerTodos, obtenerPorId, crear, actualizar, desactivar y obtenerCategoriasDisponibles.
equipos/index.php	Tabla premium compacta con Equipo, Contacto, Categoria/Torneo, Estado y Acciones.
equipos/form.php	Formulario crear/editar con estado Activo/Inactivo.
equipos/show.php	Detalle visual del equipo.

Decision importante

No se usa DELETE fisico. El estado se maneja con activo = 0/1 desde el formulario de edicion.

6. CRUD de Jugadores

Despues de Equipos se implemento Jugadores, porque los eventos de cedula necesitan jugadores asociados a equipo y posicion.

- Se crearon **Jugador.php**, **JugadorController.php** y vistas **jugadores/index.php**, **form.php** y **show.php**.
- El campo **numJug** se genera automaticamente con formato J001, J002, etc.
- El formulario carga equipos disponibles y posiciones desde SQL Server.

- Validaciones: nombre requerido, equipo, posicion, email valido, camiseta 1-999, edad >= 0 y fecha valida.
- El estado Activo/Inactivo se maneja desde editar, sin borrado fisico en el listado.

7. CRUD de Arbitros

El CRUD de Arbitros deja listo el catalogo necesario para cédulas arbitrales y para el login de arbitros.

- Se crearon **Arbitro.php**, **ArbitroController.php** y vistas para listado, formulario y detalle.
- El campo **numArb** se genera automaticamente con formato A001, A002, etc.
- Validaciones: nombre obligatorio y correo valido si se captura.
- El estado Activo/Inactivo se maneja desde editar.
- Se confirmo conceptualmente la relacion: un arbitro puede estar asociado a muchas cedulas arbitrales mediante **CedulaArbital.numArb**.

8. Modulo de Partidos y Cedulas Arbitrales

Este modulo es central para la practica. Se implemento un MVC inicial para listar partidos, ver detalle, capturar cedula y registrar eventos.

Elemento	Implementacion
Modelo	Partido.php con consultas a Partido, Jornada, Equipo, Arbitro, CedulaArbital, EventoPartido y TipoEventoPartido.
Controlador	PartidoController.php con index, show, capturarCedula, guardarCedula, agregarEvento y guardarEvento.
Vistas	partidos/index.php, partidos/show.php, cedulas/form.php, cedulas/evento_form.php y cedulas/show.php.
Stored procedure cedula	dbo.sp_registrar_cedula_arbital para crear cedula, actualizar partido y usar transaccion.
Stored procedure evento	dbo.sp_agregar_evento_cedula para agregar gol, autogol, amarilla, roja u observacion.

Pendiente de prueba real

Se deben probar desde la UI: capturar una cedula en un partido sin cedula, agregar GOL y AMARILLA, y validar en SQL Server que se inserten los registros.

9. Login y roles de aplicacion

Se implemento autenticacion basica para separar roles de aplicacion de los roles SQL Server. Esto prepara el requisito de login de arbitro para capturar/consultar cedulas.

Archivo / tabla	Funcion
database/08_auth_schema.sql	Crea dbo.Usuario y dbo.UsuarioArbitro.
Usuario	Guarda nombreUsuario, email, passwordHash, rol, activo y fechaAlta.
UsuarioArbitro	Relaciona un usuario tipo ARBITRO con un registro de dbo.Arbitro.
Usuario.php	Obtiene usuario por email, verifica password con password_verify y obtiene arbitro relacionado.
AuthController.php	Login, logout, sesion y redireccion.

Archivo / tabla	Funcion
auth.php	Helpers auth_check, auth_user, auth_role, auth_require y auth_num_arbitro.
login.php	Vista premium de inicio de sesion.
tools/generar_hash.php	Genera password_hash desde CLI sin guardar contraseñas.

Roles iniciales de aplicacion: **GERENTE**, **ASISTENTE** y **ARBITRO**. El resto de modulos queda protegido por sesion, y login/logout ya existen como rutas publicas.

10. Relacion con las practicas

Practica	Estado actual
Practica 2	Muy avanzada: indices, roles SQL iniciados, procedures, transacciones, app PHP, catalogos, cedulas y login base. Falta evidencia final, pruebas manuales y restriccion fina por arbitro.
Practica 3	Pendiente: respaldos, recuperacion, migracion, filegroups, particiones, mirroring y replicacion sobre SIGELFA_DB.
Practica 4	Pendiente: auditoria SQL Server sobre acciones criticas como cedulas, eventos, partidos, usuarios y cambios de roles.

11. Estado actual del proyecto

Bloque	Estado
Conexion PHP + SQL Server	Funcionando con PDO_SQLSRV y .env.
Base visual	Implementada y corregida contra overflow global.
CRUD Equipos	Implementado; requiere pruebas/commit si no se ha cerrado.
CRUD Jugadores	Implementado; requiere pruebas/commit si no se ha cerrado.
CRUD Arbitros	Implementado; requiere pruebas/commit si no se ha cerrado.
Partidos/Cedulas	Implementado inicialmente; requiere prueba real de guardado desde UI.
Login/Roles	Implementado inicialmente; requiere ejecutar 08_auth_schema.sql, crear usuarios y probar login/logout.

12. Checklist inmediato

- 1 Ejecutar **database/08_auth_schema.sql** en SQL Server.
- 2 Generar hash con php `tools/generar_hash.php "PasswordTemporal123"`.
- 3 Crear un usuario GERENTE de prueba manualmente en dbo.Usuario.
- 4 Probar login incorrecto y login correcto.
- 5 Crear usuario ARBITRO y relacionarlo con A001 en dbo.UsuarioArbitro.
- 6 Probar capturar cedula desde UI en un partido sin cedula.
- 7 Probar agregar eventos a una cedula usando jugadores de Halcones FC vs Tigres Laguna si se requiere data existente.

- 8 Verificar en SQL Server tablas CedulaArbitral y EventoPartido.
- 9 Revisar git status para confirmar que no se suben .env ni archivos de referencia.
- 10 Hacer commits por modulo y PR hacia main.

13. Comandos de verificacion sugeridos

Verificacion PHP:

```
php -l public/index.php
php -l app/controllers/AuthController.php
php -l app/models/Usuario.php
php -l app/helpers/auth.php
php -S localhost:8000 -t public
```

Verificacion SQL de login:

```
USE SIGELFA_DB;
GO
SELECT idUsuario, nombreUsuario, email, rol, activo FROM dbo.Usuario;
SELECT idUsuario, numArb FROM dbo.UsuarioArbitro;
GO
```

Verificacion SQL de cedulas:

```
USE SIGELFA_DB;
GO
SELECT ca.idCedula, ca.idPartido, ca.numArb, ca.estadoCedula, p.golesEquipoA, p.golesEquipoB,
p.estadoPartido
FROM dbo.CedulaArbitral ca
INNER JOIN dbo.Partido p ON ca.idPartido = p.idPartido
ORDER BY ca.idCedula DESC;

SELECT ep.idEvento, ep.idCedula, te.nombreEvento, ep.numJug, ep.minuto, ep.observacion
FROM dbo.EventoPartido ep
INNER JOIN dbo.TipoEventoPartido te ON ep.idTipoEvento = te.idTipoEvento
ORDER BY ep.idEvento DESC;
GO
```

14. Riesgos y pendientes tecnicos

Riesgo / pendiente	Accion recomendada
Pruebas manuales no ejecutadas en todos los modulos	Antes de PR, crear/editar/ver un registro por modulo y verificar en SQL Server.
Login puede bloquear rutas si no hay usuarios	Ejecutar 08_auth_schema.sql y crear al menos un GERENTE antes de probar.
ARBITRO aun no restringido a sus propias cedulas	Agregar filtro por auth_num_arbitro() en modulo de partidos/cedulas.
Cédula/eventos requieren jugadores del partido	Probar con Halcones FC vs Tigres Laguna o agregar jugadores a otros equipos.
Evidencias de practicas pendientes	Crear docs con capturas, orden de ejecucion, resultados SQL y pruebas de UI.

15. Siguiente paso recomendado

El siguiente paso debe ser cerrar pruebas de login y cédulas, no abrir mas CRUDs. Lo mas alineado con las practicas es:

-
- 1 Probar login GERENTE y ARBITRO.
 - 2 Restringir el rol ARBITRO para que solo vea/capture cedulas de su numArb.
 - 3 Probar stored procedures desde la UI y guardar evidencia.
 - 4 Preparar documento de Practica 2 con capturas y explicaciones.
 - 5 Despues pasar a Practica 3: respaldos, recuperacion, migracion, filegroups/particiones, mirroring/replicacion.
 - 6 Finalmente Practica 4: auditoria SQL Server sobre acciones criticas.